

Track Service

Automatic scheduling of railway night maintenance · Railways · pure software

Turns a night of manual spreadsheet juggling into a one-second, best-possible plan.

THE PROBLEM

Every night, a few stretches of track shut to trains for a few hours so repairs can happen. Many teams — track, signalling, electrical, bridges — all want those short slots. Today one engineer sorts it by hand on spreadsheets and phone calls: it's slow, and important jobs get dropped by accident.

WHAT OUR SOLUTION DOES

- 1 Fits in as much as possible**, prioritising urgent, safety-critical work.
- 2 Explains what didn't fit — and why** ("no crew free", "no slot long enough"), proven by re-solving, not guessed.
- 3 Handles surprises**. A 2 a.m. emergency? It re-plans instantly and shows exactly which jobs get pushed to make room.

THE THREE ASKS — ALL ANSWERED

- ✓ **Detect conflicts** — across sections, crews, machines, duty hours.
- ✓ **Optimise the plan** — maximise priority scheduled, provably.
- ✓ **Propose alternatives** — the leg most teams skip; we do it live.

Built on Google OR-Tools (CP-SAT), benchmarked against a real first-come-first-served scheduler so the gain is measurable.

THE NUMBERS TO QUOTE

34/40

JOBS SCHEDULED
greedy: 30

120

PRIORITY WEIGHT
+13 vs greedy

0.02s

SOLVE TIME
OPTIMAL, proven

+16

AVG WEIGHT GAP
5 seeds, never loses

The story isn't the count — it's *which* jobs. Greedy drops important work by luck of ordering; our engine drops only the lowest-priority work, and proves it's optimal.

RUN IT

```
# build + open dashboard (no server)
uv run trackservice run
# proof it generalises
uv run trackservice sweep
# live mode — judge invents an emergency
uv run trackservice serve
```

HARDWARE

None needed. This is a scheduling problem, not a sensing one — the input is data from railway systems, not sensors. That's by design, not a gap.

Deliverables: tested Python engine, self-contained HTML dashboard, live solve server, demo script.